

CS673 · ITERATION 0

SAFEDROP

Inventory Management & Checkout Service

Team 3 · Project Proposal & Planning · September 2026

Alexa Stein

Team Lead

Amber Rastella

Security Lead

Alex Picard

Configuration Lead

Mateus Silva

Requirements Lead

Orelmis Toribio

QA Lead

github.com/BUMETCS673/CS673OLF26P3

The Problem, and What We're Building

Orelmis

THE PROBLEM

Organizations lend expensive physical assets and track them in spreadsheets, email threads, or not at all.

- No reliable record of who holds which asset
- Approval decisions are informal and undocumented
- Disputes over returns cannot be settled
- Loss and shrinkage go undetected until audit

OUR SOLUTION

A full-stack web application where any organization can stand up its own inventory portal.

- Members browse, request, check out, and return
- Admins manage inventory and configure approval rules
- Granular, role-based access restrictions
- Every action written to a full audit history

CONCRETE EXAMPLE

A university lends 500 laptops to students for the semester. Who has unit #247 right now? Who approved it? Was it returned, and in what condition? Today that answer lives in someone's inbox. In our system it is one query against an immutable log.

Related Work — and How We Differ

Orelmis

Freshservice

IT service management with asset tracking

SIMILAR

Tracks which users hold which assets, tied to a ticketing workflow.

DIFFERENT

Asset management is secondary to ticketing. We make tracking the core product.

Libby

Library e-book lending

SIMILAR

Manages requests, approvals, checkouts, returns, and borrowing history.

DIFFERENT

Approvals depend only on license availability. Ours are configurable per item and per user tier.

Point of Rental

Equipment rental platform

SIMILAR

Inventory tracking, downtime tracking, and customer-facing checkout.

DIFFERENT

Vertical-specific and \$500+/month. Ours is general-purpose and configurable by the organization.

Our niche: audit-first, general-purpose, and configurable — not tied to one industry or one approval rule.

Essential Features & Estimates

Mateus

Portal creation & user UI

As an Org Admin, I want to create a portal so I can manage my organization's assets and equipment.

15 h

Check out / return items

As a member, I want to check out and return items so I can borrow what I need.

15 h

Login & authentication

As a member, I want to log into my dashboard so I can check the status of my requests.

12 h

Audit log

As an Org Admin, I want to see who requested and returned items, and which admins changed inventory records.

8 h

Admin dashboard

As an Org Admin, I want to add, edit, and retire assets from a single management view.

5 h

55 person-hours estimated for the essential set — the working prototype everything else builds on.

Desirable & Optional Features

Mateus

DESIRABLE — ITER. 2

- Admin-configurable approval workflows, set globally or per item
- Privileged user tiers and user groups
- Equipment groups that require elevated permissions
- AI-assisted search and product recommendations

OPTIONAL — ITER. 3

- AI reliability scoring from a borrower's return history
- AI projection of asset depreciation and future value
- JavaScript to TypeScript conversion for type safety
- Purchase-order request and approval workflow

Prioritized so that a working product exists after Iteration 1 — later iterations add value, never viability.

Nonfunctional Requirements

Amber

14

Security
requirements

14

Other
NFRs

8

Quality
categories

Coverage

- Reliability & availability · Scalability
- Performance · Compatibility
- Maintainability · Usability & accessibility
- Compliance — FERPA-aware, data minimization

Why the IDs matter

Every requirement carries a stable identifier — SR-# for security, NFR-# for everything else.

Test plan

Each SR/NFR maps to a test case

Risk register

Risks reference the requirement they threaten

Code review

Reviewers cite the ID a change satisfies

Iteration demos

We show which IDs are now met

Security by Design

Amber

Access Control

SR-1, SR-2

- RBAC across Member, Approver, and Org Admin
- Enforced server-side on every request — not in the UI
- Tenant isolation holds against direct API calls
- No cross-org read or write of inventory, requests, or logs

Audit Integrity

SR-8 – SR-10

- Append-only; no role can edit or delete history
- Actor, timestamp, before/after on every change
- Covers requests, approvals, denials, checkouts, returns
- Detailed enough for non-repudiation in disputes

Identity & Transport

SR-3 – SR-7

- bcrypt or argon2 hashing with a per-user salt
- Short-lived JWTs, httpOnly cookies, 30-min timeout
- HTTPS only, with no fallback to HTTP
- Server-side validation vs. NoSQL injection and stored XSS

Operational

SR-11 – SR-14

- Secrets in env vars or a manager, never in source control
- npm audit and CodeQL run inside the CI pipeline
- Rate-limited authentication endpoints
- CORS restricted to whitelisted frontend origins

Threats we are designing against: cross-tenant data access, NoSQL operator injection, stored XSS, brute-force login, and tampering with the audit trail.

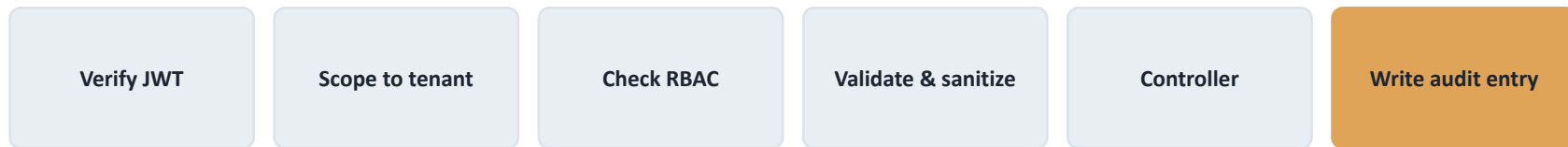
Candidate Architecture

Alex



All traffic over HTTPS / TLS

Every API request passes through the same middleware chain



Security is structural, not sprinkled — tenant scoping and RBAC sit in the request path, so no route can accidentally skip them.

Configuration Management & Tooling

Alex

Application

- React + JavaScript
- Express + Node.js
- MongoDB Atlas
- Docker + Docker Compose

Engineering

- Git / GitHub
- Jira
- GitHub Actions
- Vercel + Render

Security & AI

- GitHub CodeQL (SAST)
- OWASP ZAP (DAST)
- npm audit
- Microsoft Foundry (AI)

Branching strategy: GitHub Flow

Main stays permanently releasable. Work happens on a short-lived feature branch, opens as a pull request, and requires review from a second team member before merge. All open pull requests are merged or closed by the end of each iteration.

Selected for a five-person team on a six-week schedule — small enough to learn fast, standard enough to be defensible.

CI/CD Pipeline

Alex

1

Push / PR

Feature branch
pushed to GitHub

2

Automated checks

Lint, unit tests,
CodeQL, npm audit

3

Peer review

Second member
approves the PR

4

Merge to main

Auto-deploy to
the staging env

5

Tagged release

Promoted to
production

WHAT'S ALREADY STANDING

Repository with code / and documents / subfolders · GitHub Actions workflow running on every push · Branch protection enforced on main · Jira board loaded with the Iteration 1 backlog

OWASP ZAP runs against staging before each release is promoted.

Quality Assurance Plan

Orelmis

Metrics we will report each iteration

- Line coverage — target $\geq 80\%$
- Test case pass rate
- Average LOC per file
- Average LOC per function
- Average fan-out per file (coupling)

Testing approach

- Frontend units — Vitest + React Testing Library
- Backend units — Vitest + Supertest
- Tests written before a branch is merged
- End-to-end — Playwright, if time allows
- Happy paths and failure paths both covered

Code review

Reviewer checks logic, test coverage of the new behavior, error handling, and modularity before approving.

Defect management

Whoever finds a defect files it in Jira with reproduction steps; ownership is assigned at the weekly meeting.

Every metric is quantifiable and reported in the iteration summary — not aspirational language.

Timeline & Project Management

Alexa

Iteration 1

Essential

Database schema, backend infrastructure, authentication, CI/CD, basic UI

70 h

Iteration 2

Desirable

Approval workflows, user tiers, customer-focused enhancements

65 h

Iteration 3

Optional

AI features and additional security hardening

40 h

Cadence

Weekly team meeting on Discord, plus a second review meeting before every deliverable · Rotating minutes taker · Jira board for task assignment · Everyone appears in every iteration video

175 h

total planned

Risk Management

Amber

22 risks logged across 8 categories · Priority = $(6-L) \times (6-I) \times C$ · lower score = more urgent

12	Insufficient test coverage	Testing	Minimum coverage target on core workflows; testing time budgeted into every sprint.
12	Unfamiliarity with Git workflows	Competence	One agreed branching strategy, walked through early so every member's setup matches.
18	Integration & deployment left too late	Deployment	Integrate and deploy continuously each iteration rather than only at the end.
18	Scope creep mid-iteration	Requirements	Scope frozen at sprint start; new requests logged to the backlog for a later iteration.
18	Ambiguous requirements	Requirements	Acceptance criteria required on every story before it enters a sprint.

The register is a living document — reviewed and re-scored at every iteration boundary, not written once and filed.

AI Usage & Disclosure

Alexa

In the process

- Drafting and reviewing documentation
- Code generation and debugging support
- Test case generation
- Comparing tool and library options

In the product

- AI-assisted search and recommendations
- Borrower reliability scoring from return history
- Asset depreciation projection
- Built on Microsoft Foundry

How we disclose

- Per-student usage log, one row per task
- AI contribution table in the SPPP
- Prompt, human edits, and disposition recorded
- Nothing used without review and verification

Our standard: AI accelerates the work, but every output is reviewed, verified, and edited by the person who owns that deliverable — and that review is what gets logged, not just the prompt.

NEXT: ITERATION 1

What we deliver in two weeks

Data model

Multi-tenant schema, indexed by organization

Authentication

Login, sessions, role assignment

Core workflow

Request, approve, check out, return

Pipeline live

Tests, CodeQL, and staging deploys running

Repository

github.com/BUMETCS673/CS673OLF26P3

Project board

<https://bu-team-x3gl1rkg.atlassian.net/jira/software/projects/SCRUM/boards/1>

Thank you — questions welcome in the discussion thread.